

Deploy Magento 2 bằng Jenkins - Phần 2

Jenkins chung VPS + GitHub Webhook that

Thang 9, 2026

Deploy Magento 2 bằng Jenkins — Phần 2: Jenkins chung VPS Magento + GitHub Webhook thật

Bối cảnh: Sau giai đoạn học với Jenkins chạy trên máy local (Phần 1), giờ chuyển Jenkins sang chạy trực tiếp trên VPS đang chạy Magento 2 (Ubuntu 22.04, nginx/PHP-FPM/OpenSearch cài native). Tạo subdomain riêng cho Jenkins, kết nối GitHub Webhook để build/deploy tự động ngay khi push code — không cần chờ Poll SCM.

1. Quy ước placeholder

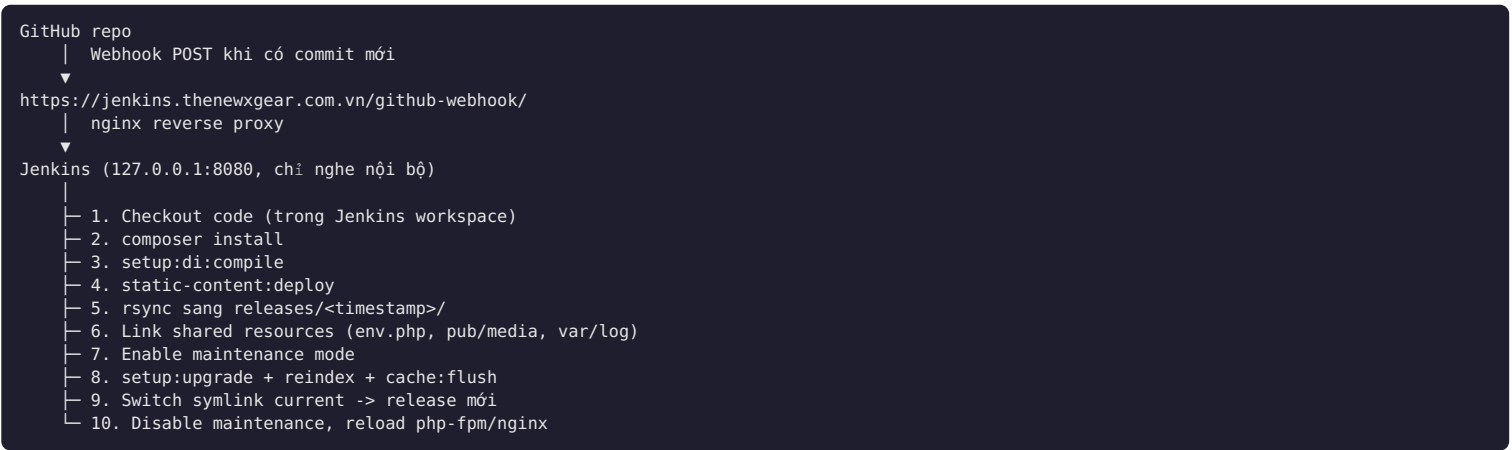
Placeholder	Ý nghĩa	Giá trị thực tế của bạn
<VPS_APP_IP>	IP VPS chạy Magento + Jenkins	IP VPS thật
<DOMAIN>	Domain chính	thenewxgear.com.vn
<JENKINS_SUBDOMAIN>	Subdomain dành cho Jenkins	jenkins.thenewxgear.com.vn
<PHP_FPM_SERVICE>	Tên service PHP-FPM	ví dụ php8.1-fpm
<GIT_REPO_URL>	URL repository Git	https://github.com/<user>/<repo>.git

2. Những gì thay đổi so với Phần 1 (Jenkins local)

Phần 1 (local + Poll SCM)	Phần 2 (chung VPS + Webhook)
Jenkins ở máy local	Jenkins cài ngay trên VPS Magento
Build local → scp / ssh sang VPS	Build và deploy chạy lệnh nội bộ, không cần SSH
Cần SSH key riêng (sshagent)	Không cần SSH key — bỏ hẳn sshagent trong Jenkinsfile
Trigger: Poll SCM mỗi vài phút	Trigger: GitHub gọi webhook — build gần như tức thời
Không cần public port	Cần expose Jenkins qua subdomain + SSL an toàn

Cấu trúc releases/shared/current trên VPS giữ nguyên 100% — không đổi gì ở phần Magento, chỉ đổi cách Jenkins tương tác với filesystem (nội bộ thay vì qua SSH).

3. Kiến trúc tổng thể



Lưu ý về thuật ngữ: pipeline này là **minimal-downtime deployment**, không phải zero-downtime tuyệt đối. Vẫn cần một khoảng

bảo trì ngắn (vài giây đến vài chục giây) trong lúc chạy `setup:upgrade`, vì code mới và schema database phải đồng bộ cùng lúc.

4. Cài Jenkins trên VPS Magento

4.1. Cài Jenkins

```
sudo apt update
sudo apt install -y openjdk-17-jre

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Jenkins mặc định chạy port 8080, chỉ truy cập tạm qua SSH tunnel trong lúc cài đặt ban đầu (sẽ khóa hẳn port này ở Bước 8):

```
# Chạy trên máy cá nhân
ssh -L 8080:localhost:8080 root@VPS_APP_IP>
```

Mở `http://localhost:8080` trên máy cá nhân, lấy password khởi tạo:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Cài **Suggested plugins** → tạo admin user.

4.2. Cài thêm plugin cần thiết

Manage Jenkins → **Plugins** → **Available plugins**, tick chọn: - **Git** - **Pipeline** - **Credentials Binding** - **GitHub** (plugin tên chính xác “GitHub Plugin” — cung cấp endpoint `/github-webhook/` và trigger “GitHub hook trigger for GITScm polling”)

Không cần **SSH Agent** plugin — vì Jenkins và Magento chung 1 máy, mọi lệnh chạy trực tiếp (`sh`), không cần `sshagent` / `ssh` / `scp`.

Restart Jenkins sau khi cài xong plugin nếu được yêu cầu.

5. Cấu hình quyền cho user `jenkins`

Jenkins chạy dưới user hệ thống tên `jenkins` (tự tạo khi cài qua apt) — cần quyền đọc/ghi vào thư mục Magento.

```
# Kiểm tra owner hiện tại
ls -la /var/www/xgear-deploy

# Thêm jenkins vào group www-data
sudo usermod -aG www-data jenkins

# Đổi owner thư mục deploy sang jenkins (KHÔNG phải "deploy" -
# vì user "deploy" chỉ dùng cho kịch bản SSH ở Phần 1, không tồn tại vai trò ở đây)
sudo chown -R jenkins:www-data /var/www/xgear-deploy
sudo chmod -R g+rwX /var/www/xgear-deploy

sudo systemctl restart jenkins
```

Cấp sudo giới hạn cho user `jenkins`

```
sudo visudo -f /etc/sudoers.d/jenkins-deploy
```

```
jenkins ALL=(ALL) NOPASSWD: /usr/sbin/service php8.1-fpm reload
jenkins ALL=(ALL) NOPASSWD: /usr/bin/systemctl reload nginx
jenkins ALL=(ALL) NOPASSWD: /usr/bin/chown -R www-data\:www-data /var/www/xgear-deploy/releases/*
```

(Sửa `php8.1-fpm` đúng version PHP đang dùng.)

Kiểm tra Jenkins gọi được PHP/Composer

```
sudo -u jenkins php -v
sudo -u jenkins composer -V
```

Nếu lỗi `command not found`, thêm PATH cho Jenkins:

```
sudo systemctl edit jenkins
```

```
[Service]
Environment="PATH=/usr/local/bin:/usr/bin:/bin"
```

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

Kiểm chứng quyền ghi thực tế

```
sudo -u jenkins touch /var/www/xgear-deploy/releases/test.txt \
&& echo "OK - ghi được" \
&& sudo -u jenkins rm /var/www/xgear-deploy/releases/test.txt
```

6. Tạo subdomain + SSL cho Jenkins

6.1. Tạo DNS record trên Cloudflare

Cloudflare Dashboard → chọn domain → DNS → Records → Add record:

```
Type:      A
Name:      jenkins
IPv4 address: <VPS_APP_IP>
Proxy status: Proxied (đám mây cam)
TTL:      Auto
```

Kiểm tra DNS đã lan truyền:

```
dig jenkins.thenewxgear.com.vn +short
```

Nếu gặp vướng mắc khi xin SSL do Cloudflare Proxy, tạm chuyển sang **DNS only** (đám mây xám) trong lúc chạy Certbot, bật lại Proxy sau khi xong.

6.2. Tạo file nginx config — CHỈ phần HTTP trước

Thứ tự đúng: tạo config HTTP (port 80) trước, chạy Certbot sau — không làm ngược lại, nếu không Certbot sẽ báo lỗi “Could not automatically find a matching server block”.

```
sudo nano /etc/nginx/sites-available/jenkins
```

```
server {
    listen 80;
    server_name jenkins.thenewxgear.com.vn;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/jenkins /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Kiểm tra trước khi chạy Certbot:

```
curl -I http://jenkins.thenewxgear.com.vn
```

(Nhận được phản hồi bất kỳ, kể cả lỗi 502, là đủ — mục đích chỉ xác nhận nginx đã nhận diện đúng domain.)

6.3. Chạy Certbot xin SSL

```
sudo certbot --nginx -d jenkins.thenewxgear.com.vn
```

Kết quả mong đợi:

```
Successfully received certificate.
Deploying certificate
Successfully deployed certificate for jenkins.thenewxgear.com.vn ...
```

Certbot sẽ tự động sửa file `/etc/nginx/sites-available/jenkins`, thêm các dòng `ssl_certificate`, `ssl_certificate_key`, và tạo thêm server block redirect HTTP → HTTPS.

6.4. Kiểm tra file config sau khi Certbot tự sửa

```
sudo cat /etc/nginx/sites-available/jenkins
```

Kết quả đúng dạng:

```
server {
    server_name jenkins.thenewxgear.com.vn;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparam.pem; # managed by Certbot
}

server {
    if ($host = jenkins.thenewxgear.com.vn) {
        return 301 https://$host$request_uri;
    } # managed by Certbot

    listen 80;
    server_name jenkins.thenewxgear.com.vn;
    return 404; # managed by Certbot
}
```

```
sudo nginx -t
sudo systemctl reload nginx
curl -I https://jenkins.thenewxgear.com.vn
```

Phải trả về `HTTP/2 200` hoặc `HTTP/2 302`. Mở trình duyệt kiểm tra ổ khóa SSL hợp lệ.

6.5. Xác nhận renew tự động

```
sudo certbot certificates
sudo certbot renew --dry-run
```

7. Hai lựa chọn bảo mật UI Jenkins — chọn 1 trong 2

OPTION A — Không dùng Basic Auth (đơn giản, phù hợp giai đoạn học)

Giữ nguyên file nginx config như Bước 6.4, không sửa gì thêm. Bảo mật dựa vào: - Jenkins vẫn có trang login riêng - Port 8080 chỉ bind `127.0.0.1` (Bước 8), không truy cập trực tiếp được từ ngoài

Ưu điểm: đơn giản, không cần quản lý thêm file password. **Nhược điểm:** endpoint `/github-webhook/` mở public hoàn toàn — ai cũng gửi POST giả kích hoạt build được (dù không đọc được kết quả build vì vẫn cần login Jenkins).

Bỏ qua sang Bước 8 nếu chọn Option A.

OPTION B — Dùng Basic Auth (khuyến nghị cho môi trường thật sự production)

7.1. Cài công cụ tạo file password

```
sudo apt install -y apache2-utils
```

7.2. Tạo user/password bảo vệ

```
sudo htpasswd -c /etc/nginx/.htpasswd youradmin
```

(Đổi `youradmin` theo ý muốn, nhập password khi được hỏi.)

Lưu ý cờ `-c`: chỉ dùng `-c` lần đầu tạo file. Thêm user thứ 2 về sau thì **bỏ `-c`**:

```
sudo htpasswd /etc/nginx/.htpasswd second_user
```

Dùng lại `-c` sẽ ghi đè, xóa mất user cũ.

7.3. Sửa nginx config — thêm Basic Auth, loại trừ path webhook

```
sudo nano /etc/nginx/sites-available/jenkins
```

```
server {
    server_name jenkins.thenewxgear.com.vn;

    # Webhook GitHub - KHÔNG yêu cầu Basic Auth (GitHub không tu nhập được)
    location /github-webhook/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # UI Jenkins - YÊU CẦU Basic Auth
    location / {
        auth_basic "Restricted Area";
        auth_basic_user_file /etc/nginx/.htpasswd;

        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect http:// https://;
        proxy_read_timeout 90s;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparam.pem; # managed by Certbot
}

server {
    if ($host = jenkins.thenewxgear.com.vn) {
        return 301 https://$host$request_uri;
    } # managed by Certbot

    listen 80;
    server_name jenkins.thenewxgear.com.vn;
    return 404; # managed by Certbot
}
```

Về thứ tự location: nginx tự ưu tiên khớp path cụ thể/dài hơn (`/github-webhook/`) so với path gốc (`/`) theo nguyên tắc “prefix match” — không quan trọng khai báo trước hay sau trong file, nhưng nên viết `/github-webhook/` trước cho dễ đọc.

```
sudo nginx -t
sudo systemctl reload nginx
```

7.4. Test Basic Auth

```
# UI Jenkins - phải bị chặn hỏi password
curl -I https://jenkins.thenewxgear.com.vn/

# Webhook path - KHÔNG bị hỏi password
curl -I https://jenkins.thenewxgear.com.vn/github-webhook/
```

Dòng đầu phải trả 401. Dòng hai trả mã khác 401 (thường 403 / 405 vì Jenkins từ chối GET, chỉ nhận POST — đây là bình thường).

8. Chặn truy cập trực tiếp vào port 8080

Dù chọn Option A hay B, **bắt buộc** làm bước này — nếu không, ai cũng có thể bỏ qua toàn bộ nginx bằng cách gõ thẳng `http://<VPS_APP_IP>:8080`.

```
sudo systemctl edit jenkins
```

```
[Service]
Environment="JENKINS_LISTEN_ADDRESS=127.0.0.1"
```

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

Kiểm tra:

```
sudo ss -tlnp | grep 8080
```

Phải hiện `127.0.0.1:8080`, **không được** hiện `0.0.0.0:8080`.

Kiểm tra từ máy ngoài (không phải VPS):

```
curl -I http://<VPS_APP_IP>:8080 --connect-timeout 5
```

Phải bị “Connection refused” hoặc timeout.

Rà soát thêm firewall:

```
sudo ufw status
sudo ufw delete allow 8080 # chỉ chạy nếu trước đó có rule này
sudo ufw allow 443
sudo ufw allow 80
sudo ufw allow 22
```

9. Kết nối Jenkins với Git repo

9.1. Tạo credential Git (nếu repo private)

GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → scope `repo` → copy token.

Jenkins: Manage Jenkins → Credentials → System → Global credentials → Add Credentials: - Kind: Username with password - Username: username GitHub - Password: personal access token - ID: `git-xgear-token`

9.2. Tạo Pipeline Job

1. **New Item** → tên `deploy-xgear-vps` → chọn **Pipeline** → OK
2. **Pipeline → Definition:** `Pipeline script from SCM`
 - SCM: `Git`
 - Repository URL: `<GIT_REPO_URL>`
 - Credentials: `git-xgear-token`
 - Branches to build: `*/main`
 - Script Path: `Jenkinsfile`

10. Jenkinsfile hoàn chỉnh (bản chung VPS, không cần SSH)

```
pipeline {
  agent any
```

```

environment {
    DEPLOY_BASE      = '/var/www/xgear-deploy'
    RELEASE_NAME     = "${new Date().format('yyyyMMddHHmmss')}"
    RELEASE_PATH     = "${DEPLOY_BASE}/releases/${RELEASE_NAME}"
    KEEP_RELEASES    = 5
    PHP_FPM_SERVICE  = 'php8.1-fpm'
}

options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '20'))
}

stages {

    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Composer Install') {
        steps {
            sh 'composer install --no-dev --optimize-autoloader --prefer-dist'
        }
    }

    stage('Chuan bi env.php gia de compile') {
        steps {
            sh 'cp app/etc/env.php.jenkins app/etc/env.php'
        }
    }

    stage('DI Compile') {
        steps {
            sh 'php bin/magento setup:di:compile'
        }
    }

    stage('Static Content Deploy') {
        steps {
            sh 'php bin/magento setup:static-content:deploy -f vi_VN en_US'
        }
    }

    stage('Copy vao thu muc release moi') {
        steps {
            sh """
                mkdir -p ${RELEASE_PATH}
                rsync -a --exclude='.git' ./ ${RELEASE_PATH}/
            """
        }
    }

    stage('Link Shared Resources') {
        steps {
            sh """
                rm -f ${RELEASE_PATH}/app/etc/env.php
                ln -sfn ${DEPLOY_BASE}/shared/app/etc/env.php ${RELEASE_PATH}/app/etc/env.php

                rm -rf ${RELEASE_PATH}/pub/media
                ln -sfn ${DEPLOY_BASE}/shared/pub/media ${RELEASE_PATH}/pub/media

                rm -rf ${RELEASE_PATH}/var/log
                ln -sfn ${DEPLOY_BASE}/shared/var/log ${RELEASE_PATH}/var/log
            """
        }
    }

    stage('Enable Maintenance Mode') {
        steps {
            sh "cd ${RELEASE_PATH} && php bin/magento maintenance:enable"
        }
    }

    stage('DB Upgrade & Reindex') {
        steps {
            sh """
                cd ${RELEASE_PATH}
                php bin/magento setup:upgrade --keep-generated
                php bin/magento indexer:reindex
                php bin/magento cache:flush
            """
        }
    }

    stage('Switch Symlink (Go Live)') {
        steps {
            sh "ln -sfn ${RELEASE_PATH} ${DEPLOY_BASE}/current"
        }
    }

    stage('Disable Maintenance & Reload Services') {
        steps {

```

```

        sh """
            cd ${DEPLOY_BASE}/current
            php bin/magento maintenance:disable
            sudo service ${PHP_FPM_SERVICE} reload
            sudo systemctl reload nginx
        """
    }
}

stage('Fix Permissions') {
    steps {
        sh "sudo chown -R www-data:www-data ${RELEASE_PATH}"
    }
}

stage('Cleanup Old Releases') {
    steps {
        sh """
            cd ${DEPLOY_BASE}/releases
            ls -lt | tail -n +\${((${KEEP_RELEASES}+1))} | xargs -r rm -rf
        """
    }
}
}

post {
    failure {
        echo 'Deploy that bai - kiểm tra Console Output, cần nhắc rollback bằng job riêng.'
    }
    success {
        echo "Deploy thành công: release ${RELEASE_NAME}"
    }
}
}

```

Giải thích: “source folder” nằm ở đâu?

Lệnh `checkout scm` **không nhận tham số đường dẫn** — Jenkins tự động chọn workspace dựa theo tên job:

```
/var/lib/jenkins/workspace/deploy-xgear-vps/
```

Mọi lệnh `sh` phía sau trong cùng pipeline **mặc định chạy trong đúng thư mục này** — không cần `cd` tường minh. Dấu `./` trong lệnh `rsync -a ./ ${RELEASE_PATH}/` chính là workspace đó.

Thư mục này do Jenkins tự quản lý, không phải nơi bạn thao tác Git thủ công (nơi đó là thư mục bạn tự clone riêng, ví dụ `/var/www/xgear-source`, chỉ dùng để soạn `Jenkinsfile`, `env.php.jenkins`, rồi push lên GitHub — xem mục 11).

11. Chuẩn bị `env.php.jenkins` (không đổi so với Phần 1)

File này cần tồn tại sẵn trong repo trước khi Jenkins build lần đầu — tạo tại thư mục bạn tự clone (ví dụ `/var/www/xgear-source`), **không phải** Jenkins workspace:

```

cd /var/www/xgear-source
nano app/etc/env.php.jenkins

```

```
<?php
return [
    'backend' => ['frontName' => 'admin'],
    'cache' => [
        'frontend' => [
            'default' => ['backend' => 'Cm_Cache_Backend_File'],
            'page_cache' => ['backend' => 'Cm_Cache_Backend_File']
        ]
    ],
    'db' => [
        'table_prefix' => '',
        'connection' => [
            'default' => [
                'host' => '127.0.0.1',
                'dbname' => 'magento_dummy',
                'username' => 'dummy_user',
                'password' => 'dummy_pass',
                'active' => '1'
            ]
        ]
    ],
    'crypt' => ['key' => 'dummy32characterkeyforcompileonly'],
    'resource' => ['default_setup' => ['connection' => 'default']],
    'session' => ['save' => 'files'],
    'MAGE_MODE' => 'production',
    'cache_types' => [
        'config' => 1, 'layout' => 1, 'block_html' => 1, 'collections' => 1,
        'reflection' => 1, 'db_ddl' => 1, 'compiled_config' => 1, 'eav' => 1,
        'customer_notification' => 1, 'config_integration' => 1,
        'config_integration_api' => 1, 'full_page' => 1,
        'config_webservice' => 1, 'translate' => 1
    ]
]
```

Đảm bảo `.gitignore` chặn đúng `env.php` thật (không dùng wildcard):

```
/app/etc/env.php
```

Kiểm tra trước khi commit:

```
git status
git check-ignore -v app/etc/env.php
git add app/etc/env.php.jenkins .gitignore Jenkinsfile
git commit -m "Add Jenkinsfile and env.php.jenkins for VPS pipeline"
git push origin main
```

12. Cấu hình GitHub Webhook

12.1. Tạo webhook cơ bản (không dùng secret)

Vào repo GitHub → **Settings** → **Webhooks** → **Add webhook**:

Trường	Giá trị
Payload URL	<code>https://jenkins.thenewxgear.com.vn/github-webhook/</code>
Content type	<code>application/json</code>
SSL verification	Enable SSL verification
Which events	Just the push event
Active	tick

12.2. Tùy chọn nâng cao: thêm Secret cho webhook

Điền secret vào ô **Secret** trong form trên GitHub (một chuỗi ký tự ngẫu nhiên tự đặt, ví dụ tạo bằng `openssl rand -hex 20`).

Lưu ý quan trọng: chỉ điền secret ở phía GitHub là **chưa đủ** — Jenkins phải được cấu hình biết secret đó thì mới thực sự verify chữ ký HMAC (header `X-Hub-Signature-256`). Nếu bỏ qua bước cấu hình phía Jenkins, GitHub vẫn gửi kèm chữ ký nhưng Jenkins **không kiểm tra**, webhook vẫn hoạt động bình thường — chỉ là secret “không có tác dụng thật”.

Để secret có tác dụng thật, cấu hình phía Jenkins:

Bước A — Tạo Credential chứa secret: 1. Manage Jenkins → Credentials → System → Global credentials → Add Credentials 2. Kind: **Secret text** 3. Secret: dán đúng chuỗi đã điền bên GitHub 4. ID: `github-webhook-secret` 5. Create

Bước B — Gắn secret vào cấu hình GitHub plugin:

Vị trí UI có thể khác nhau tùy version Jenkins. Cách tìm chắc chắn nhất: 1. Vào **Manage Jenkins**, dùng ô **Search** ở đầu trang, gõ `GitHub` 2. Hoặc vào **Manage Jenkins → System** (một số version gọi là “Configure System”), dùng `Ctrl+F` tìm `GitHub` 3. Tìm nút **“Advanced”** (icon quyển sổ + bút chì) trong phần cấu hình GitHub Server 4. Mở rộng ra sẽ thấy ô **“Shared secrets”** → Add → chọn credential `github-webhook-secret` 5. Save

Nếu không thấy mục GitHub nào trong System Configuration, kiểm tra lại plugin đã cài đúng tên **“GitHub Plugin”** chưa (Manage Jenkins → Plugins → Installed), cài xong cần restart Jenkins.

Đây là bước tùy chọn — bỏ qua hoàn toàn không ảnh hưởng chức năng chính của webhook, chỉ thiếu lớp xác thực chữ ký chống giả mạo request.

12.3. Bật trigger trong Jenkins job

Job `deploy-xgear-vps` → **Configure** → **Build Triggers** → tick:

```
GitHub hook trigger for GITScm polling
```

Save.

13. Test webhook thật

13.1. Push commit thử

```
cd /var/www/xgear-source
echo "// test webhook $(date)" >> app/etc/env.php.jenkins
git add .
git commit -m "test: trigger webhook"
git push origin main
```

13.2. Kiểm tra trên GitHub

Repo → **Settings** → **Webhooks** → webhook vừa tạo → tab **Recent Deliveries**: - ✓ xanh + Response `200` → thành công - ✗ đỏ → xem bảng lỗi bên dưới

Response code	Nguyên nhân	Cách sửa
401	Basic Auth đang chặn nhầm path webhook (nếu chọn Option B)	Kiểm tra lại <code>location /github-webhook/</code> trong nginx config
404	Sai URL, thiếu dấu <code>/</code> cuối	Sửa lại <code>.../github-webhook/</code>
Timeout	DNS/firewall chặn	<code>dig jenkins.thenewxgear.com.vn</code> , <code>curl -I https://jenkins...</code> từ máy ngoài
502 Bad Gateway	nginx không kết nối được Jenkins	<code>systemctl status jenkins</code> , <code>ss -tlnp grep 8080</code>
403	Plugin GitHub chưa cài đúng	Kiểm tra lại Manage Jenkins → Plugins

13.3. Xác nhận Jenkins nhận build tự động

Jenkins → job `deploy-xgear-vps` → **Build History** — build mới xuất hiện trong vài giây sau khi push. Click vào → **Console Output** theo dõi pipeline.

14. Job Rollback (bản chung VPS)

```

pipeline {
  agent any
  parameters {
    string(name: 'RELEASE_TO_ROLLBACK', defaultValue: '', description: 'De trong = quay ve release lien truoc')
  }
  environment {
    DEPLOY_BASE = '/var/www/xgear-deploy'
  }
  stages {
    stage('Rollback') {
      steps {
        sh '''
          cd ${DEPLOY_BASE}/releases
          TARGET="${RELEASE_TO_ROLLBACK}"
          if [ -z "$TARGET" ]; then
            TARGET=$(ls -lt | sed -n 2p)
          fi
          echo "Rolling back to: $TARGET"
          ln -sf ${DEPLOY_BASE}/releases/$TARGET ${DEPLOY_BASE}/current
          sudo service php8.1-fpm reload
        '''
      }
    }
  }
}

```

15. Giới hạn tài nguyên Jenkins (tránh ảnh hưởng site đang chạy)

```
sudo systemctl edit jenkins
```

```

[Service]
MemoryMax=1500M
CPUQuota=100%

```

```

sudo systemctl daemon-reload
sudo systemctl restart jenkins

```

Theo dõi tài nguyên trong lúc build:

```
htop
```

16. Checklist tổng hợp trước khi vận hành thật

- ☐ Jenkins cài trên VPS, plugin Git/Pipeline/Credentials Binding/GitHub đã cài
- ☐ User `jenkins` join group `www-data`, owner `/var/www/xgear-deploy` là `jenkins:www-data`
- ☐ Sudoers cấp đúng NOPASSWD cho reload php-fpm, nginx, chown releases
- ☐ DNS subdomain `jenkins.<DOMAIN>` đã trở đúng, SSL Certbot cấp thành công
- ☐ (Option B) Basic Auth hoạt động cho `/`, không áp dụng cho `/github-webhook/`
- ☐ Port 8080 chỉ bind `127.0.0.1`, không truy cập được trực tiếp từ ngoài
- ☐ `env.php.jenkins` đã có trong repo, `.gitignore` chặn đúng `env.php` thật
- ☐ `Jenkinsfile` (bản không SSH) đã push lên repo, Script Path job trở đúng
- ☐ Webhook GitHub tạo xong, "Recent Deliveries" trả `200`
- ☐ (Tùy chọn) Secret webhook đã cấu hình đủ cả 2 phía GitHub và Jenkins nếu muốn verify thật
- ☐ Job Rollback đã tạo và test thử ít nhất 1 lần
- ☐ Jenkins đã giới hạn `MemoryMax` / `CPUQuota`

17. Xử lý lỗi thực tế: Composer 401 khi tải package `repo.magento.com`

Đây là lỗi thực tế đã gặp khi build lần đầu trên VPS, ghi lại chi tiết để tham khảo khi gặp lại.

17.1. Triệu chứng

Log dừng ở stage `Composer Install`, giữa lúc đang tải các package Braintree/PayPal:

```
- Downloading paypal/module-braintree-customer-balance (4.7.0)
```

```
In AuthHelper.php line 225:  
The 'https://repo.magento.com/archives/magento/composer-dependency-version-audit-plugin/magento-composer-dependency-version-audit-plugin-0.1.6.0.zip'  
URL required authentication (HTTP 401).  
You must be using the interactive console to authenticate
```

17.2. Nguyên nhân gốc

Các package `paypal/module-braintree-*` thuộc kho riêng của Adobe/Magento (`repo.magento.com`), không phải Packagist công khai — cần **Magento Marketplace Authentication Keys** (Public key + Private key) để tải.

Trên máy dùng để code hằng ngày, key này thường đã lưu sẵn trong `auth.json` ở home directory. Nhưng Jenkins chạy composer dưới **user hệ thống jenkins**, với home riêng biệt:

```
sudo -u jenkins composer config -g home  
# => /var/lib/jenkins/.config/composer (Composer 2.3+ dùng chuẩn XDG, không còn ~/.composer/)
```

Thư mục này trống, chưa từng có `auth.json` → Composer không biết cách xác thực với `repo.magento.com` → lỗi `401`.

17.3. Cách kiểm tra xác nhận đúng nguyên nhân

```
sudo -u jenkins composer config -g home  
sudo ls -la $(sudo -u jenkins composer config -g home)
```

Nếu không thấy file `auth.json` trong thư mục trả về — đây chính xác là nguyên nhân.

Kiểm tra key còn hợp lệ hay không bằng `curl` trực tiếp (không phụ thuộc Composer):

```
curl -u "<PUBLIC_KEY>:<PRIVATE_KEY>" -I \  
"https://repo.magento.com/archives/magento/composer-dependency-version-audit-plugin/magento-composer-dependency-version-audit-plugin-0.1.6.0.zip"
```

- `HTTP/2 200` hoặc `302` → key đúng, vấn đề nằm ở cách cấu hình chứ không phải key sai
- `HTTP/2 401` → key sai/hết hạn, cần lấy lại từ marketplace.magento.com

17.4. Cách fix — 3 phương án

Phương án 1 (khuyến nghị): Jenkins Credentials + biến môi trường `COMPOSER_AUTH` trong Jenkinsfile

Tạo credential trong Jenkins (Kind: Username with password, Username = Public key, Password = Private key, ID: `magento-marketplace-keys`), sau đó sửa stage `Composer Install`:

```
stage('Composer Install') {  
    steps {  
        withCredentials([usernamePassword(  
            credentialsId: 'magento-marketplace-keys',  
            usernameVariable: 'MAGENTO_PUBLIC_KEY',  
            passwordVariable: 'MAGENTO_PRIVATE_KEY'  
        )) {  
            sh '''  
                export COMPOSER_AUTH='{ "http-basic": { "repo.magento.com":  
{ "username": "'"$MAGENTO_PUBLIC_KEY"'", "password": "'"$MAGENTO_PRIVATE_KEY"'"} } }'  
                composer install --no-dev --optimize-autoloader --prefer-dist  
            ...  
            }  
        }  
    }  
}
```

Ưu điểm: key không nằm trần trên đĩa, không lộ trong Console Output (Jenkins tự che giá trị `withCredentials`), dễ thu hồi/đổi khi cần.

Phương án 2: Tạo `auth.json` global 1 lần thủ công trên VPS

```
sudo -u jenkins composer config -g http-basic.repo.magento.com "<PUBLIC_KEY>" "<PRIVATE_KEY>"
```

Không cần sửa Jenkinsfile, nhưng key nằm cố định plaintext trên đĩa — kém an toàn hơn Phương án 1.

Phương án 3: Tạo `auth.json` ngay trong stage compose bằng lệnh `composer config -g` (biến thể của Phương án 1)

```
stage('Cau hình Magento auth.json') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: 'magento-marketplace-keys',
      usernameVariable: 'MAGENTO_PUBLIC_KEY',
      passwordVariable: 'MAGENTO_PRIVATE_KEY'
    )]) {
      sh 'composer config -g http-basic.repo.magento.com "$MAGENTO_PUBLIC_KEY" "$MAGENTO_PRIVATE_KEY"'
    }
  }
}
```

Ghi lại `auth.json` global dùng chung cho mọi build sau, không cần export `COMPOSER_AUTH` mỗi lần.

17.5. Lỗi phát sinh tiếp theo: “Could not find credentials entry”

Sau khi sửa Jenkinsfile theo Phương án 1/3, có thể gặp lỗi mới:

```
ERROR: Could not find credentials entry with ID 'magento-marketplace-keys'
Finished: FAILURE
```

Nguyên nhân: credential được tham chiếu trong Jenkinsfile (`credentialsId: 'magento-marketplace-keys'`) nhưng **chưa thực sự được tạo** trong Jenkins Credentials Store — dễ nhầm lẫn vì việc test `composer install` thành công qua SSH thủ công (tự export biến môi trường) **hoàn toàn độc lập** với việc Jenkins Credentials Store có chứa ID đó hay không. Hai việc này không liên quan nhau về mặt hệ thống.

Cách kiểm tra chắc chắn credential đã tồn tại — dùng Script Console (đáng tin cậy hơn dò UI, vì UI khác nhau tùy version):

Vào `https://<jenkins-domain>/script`, chạy:

```
import com.cloudbees.plugins.credentials.CredentialsProvider
import com.cloudbees.plugins.credentials.common.StandardCredentials
import jenkins.model.Jenkins

def creds = CredentialsProvider.lookupCredentials(
  StandardCredentials.class,
  Jenkins.instance,
  null,
  null
)

creds.each { c -> println "ID: ${c.id} | Class: ${c.class.simpleName}" }
if (creds.isEmpty()) { println "KHÔNG CÓ CREDENTIAL NAO TON TAI TRONG JENKINS" }
```

Nếu không thấy đúng dòng `ID: magento-marketplace-keys` trong kết quả, credential chưa tồn tại — cần tạo lại.

Cách tạo credential chắc chắn — 2 lựa chọn:

Qua UI (URL trực tiếp, tránh bấm nhầm/thoát giữa chừng qua nhiều màn hình):

```
https://<jenkins-domain>/manage/credentials/store/system/domain/_/newCredentials
```

Điền Kind: `Username with password`, Username/Password là Public/Private key, ID: `magento-marketplace-keys`, bấm **Create** và xác nhận trang tự chuyển về danh sách credentials (dấu hiệu lưu thành công).

Qua Script Console (nhANH, không phụ thuộc UI):

```
import com.cloudbees.plugins.credentials.CredentialsScope
import com.cloudbees.plugins.credentials.impl.UsernamePasswordCredentialsImpl
import com.cloudbees.plugins.credentials.CredentialsProvider
import jenkins.model.Jenkins

def store = CredentialsProvider.lookupStores(Jenkins.instance).find {
  it.getDisplayName() == "System"
}

def newCred = new UsernamePasswordCredentialsImpl(
  CredentialsScope.GLOBAL,
  "magento-marketplace-keys",
  "Magento Marketplace Keys",
  "<PUBLIC_KEY>",
  "<PRIVATE_KEY>"
)

store.addCredentials(com.cloudbees.plugins.credentials.domains.Domain.global(), newCred)
println "Đã tạo xong credential: magento-marketplace-keys"
```

17.6. Cách test toàn bộ luồng sau khi fix

- Chạy lại script kiểm tra Script Console, xác nhận đủ credential `magento-marketplace-keys` xuất hiện trong danh sách.
- Vào job → **Build Now** (không bắt buộc phải có commit mới — lỗi này nằm ở tầng cấu hình Jenkins, không liên quan tới code đã push).

3. Theo dõi Console Output, xác nhận:
- Không còn `ERROR: Could not find credentials entry`
 - Stage `Composer Install` không còn `HTTP 401` khi tải `paypal/module-braintree-*`
 - Pipeline tiếp tục chạy qua các stage sau (`DI Compile` , `Static Content Deploy` , ...) thay vì bị `skipped due to earlier failure(s)` .

17.7. Lưu ý bảo mật

Vì Magento Marketplace key từng được dán trực tiếp vào kênh trao đổi (chat/log) trong lúc debug, nên **rotate (đổi) lại key này** trên marketplace.magento.com → My Profile → Access Keys sau khi mọi thứ hoạt động ổn định, rồi cập nhật giá trị mới vào Jenkins Credentials tương ứng.

18. Bảng lỗi thường gặp tổng hợp

Lỗi	Nguyên nhân	Cách sửa
<code>Could not automatically find a matching server block</code> (certbot)	Chạy certbot trước khi tạo file nginx config	Tạo config HTTP (port 80) trước, chạy certbot sau
<code>Permission denied</code> khi Jenkins ghi vào <code>releases/</code>	Owner thư mục sai (là <code>deploy</code> thay vì <code>jenkins</code>)	<code>sudo chown -R jenkins:www-data /var/www/xgear-deploy</code>
<code>sudo: no tty present</code>	Sudoers thiếu <code>NOPASSWD</code>	Kiểm tra lại <code>/etc/sudoers.d/jenkins-deploy</code>
<code>composer: command not found</code> (dưới user jenkins)	PATH của Jenkins thiếu	Thêm <code>Environment="PATH=..."</code> qua <code>systemctl edit jenkins</code>
Webhook trả <code>401</code>	Basic Auth áp dụng nhầm cho path webhook	Kiểm tra thứ tự/khai báo <code>location /github-webhook/</code>
Webhook trả <code>502</code>	Jenkins service chưa chạy hoặc sai port proxy	<code>systemctl status jenkins</code> , kiểm tra <code>proxy_pass http://127.0.0.1:8080</code>
Site lỗi 500 sau khi switch symlink	Thiếu quyền đọc hoặc thiếu <code>var/generation</code>	<code>tail -f /var/www/xgear-deploy/current/var/log/exception.log</code>
<code>HTTP 401</code> khi composer tải <code>repo.magento.com</code>	User <code>jenkins</code> chưa có Magento Marketplace auth.json	Xem chi tiết mục 17 — dùng <code>COMPOSER_AUTH</code> qua Jenkins Credentials
<code>Could not find credentials entry with ID '...'</code>	Credential tham chiếu trong Jenkinsfile chưa thực sự tồn tại trong Jenkins Credentials Store	Xem chi tiết mục 17.5 — xác nhận và tạo lại qua Script Console

Tài liệu này là Phần 2, tiếp nối Phần 1 (Jenkins chạy trên máy local với SCM Polling). Cả hai dùng chung cấu trúc `releases/shared/current` trên VPS Magento — phần khác biệt chính nằm ở cách Jenkins kết nối tới VPS (qua SSH ở Phần 1, chạy nội bộ ở Phần 2) và cách trigger build (Poll SCM ở Phần 1, Webhook tức thời ở Phần 2).